Stacks, Queues and Monotonicity

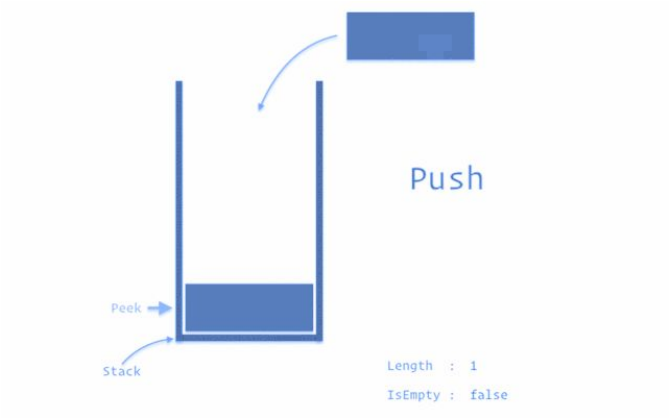
A2SV
Africa To Silicon Valley

Pre-requisites

- Linear data structures array/list
- Linked List

A2SV
Africa To Silicon Valley

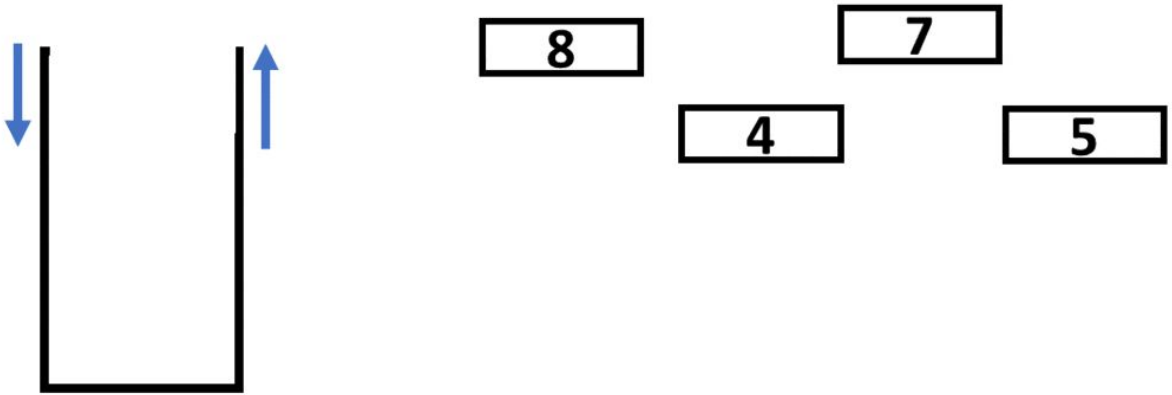
Stacks



A2SV
Africa To Silicon Valley

Introduction To Stack

Stack data structure is a linear data structure that accompanies a principle known as **LIFO** (Last In First Out) or FILO (First In Last Out).



A2SV
Africa To Silicon Valley

Real Life Example

Stack of plates

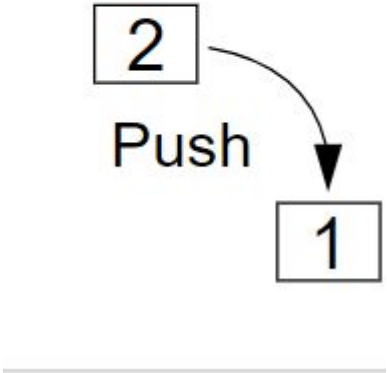
Stack of books

A2SV
Africa To Silicon Valley

Stack Operations

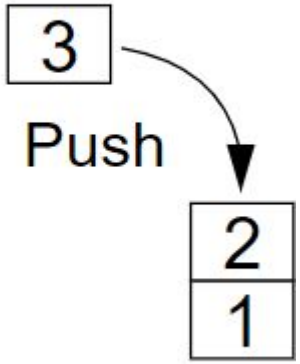
Push operation

- Add an element to the top of the stack



A2SV
Africa To Silicon Valley

Stack Operations

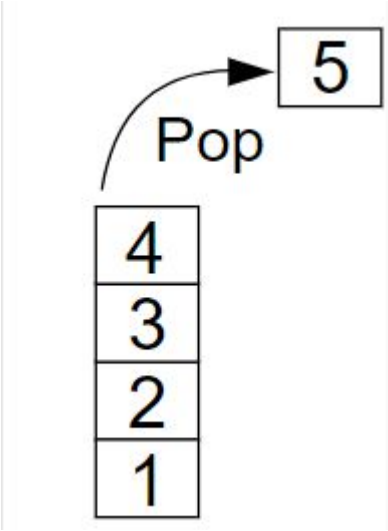


A2SV
Africa To Silicon Valley

Stack Operations

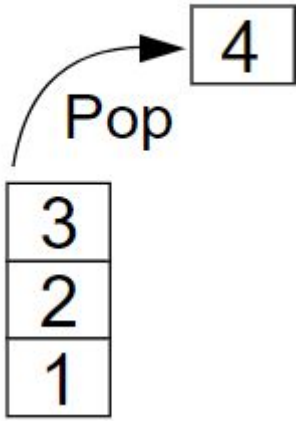
Pop operation

- Remove element from the top of the stack



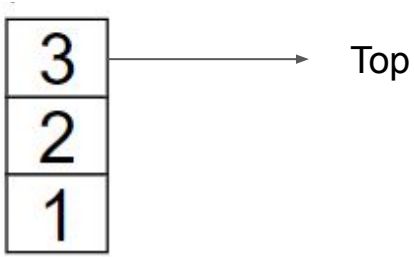
A2SV
Africa To Silicon Valley

Stack Operations



A2SV
Africa To Silicon Valley

Stack Operations



Peek operation

- returning the top element of a stack.

Is empty()

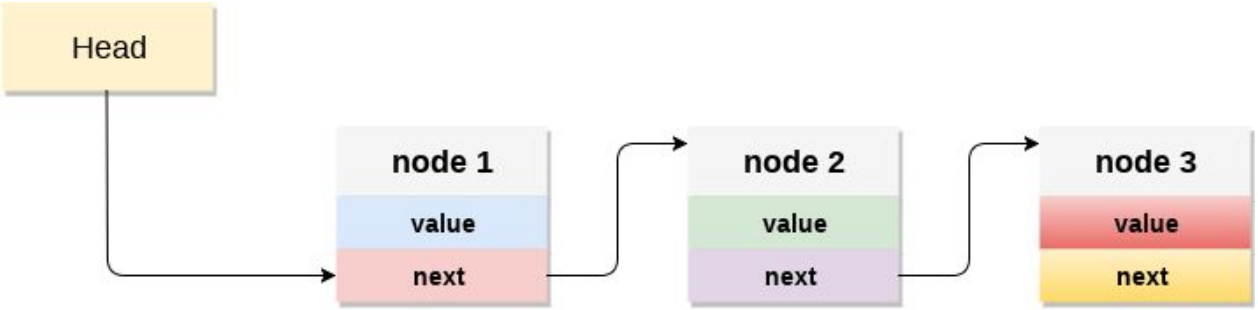
- Check if the list is empty or not.
- If it's empty return True else False.

A2SV
Africa To Silicon Valley

How can we implement Stack?

A2SV
Africa To Silicon Valley

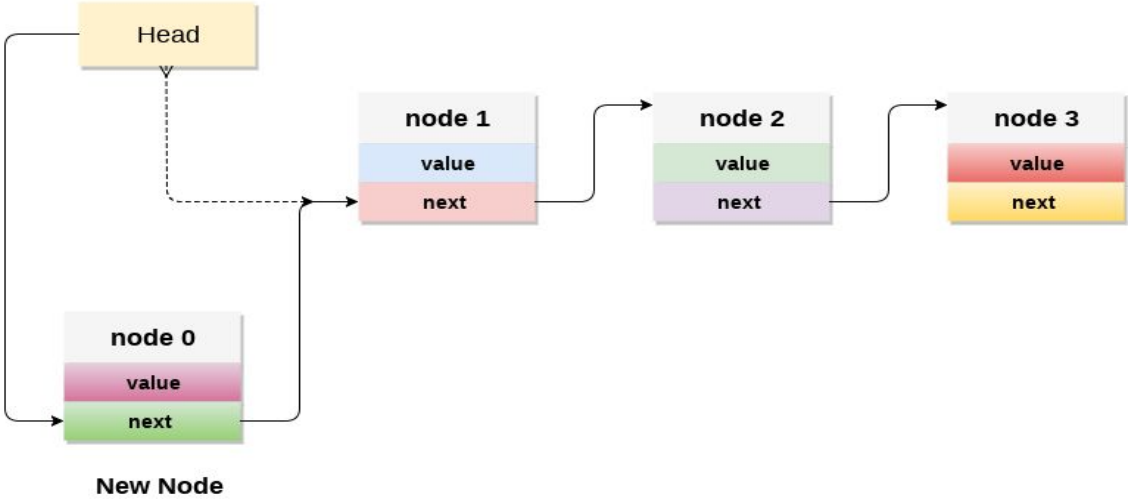
Implementing stack using linked list



A2SV
Africa To Silicon Valley

Push operation

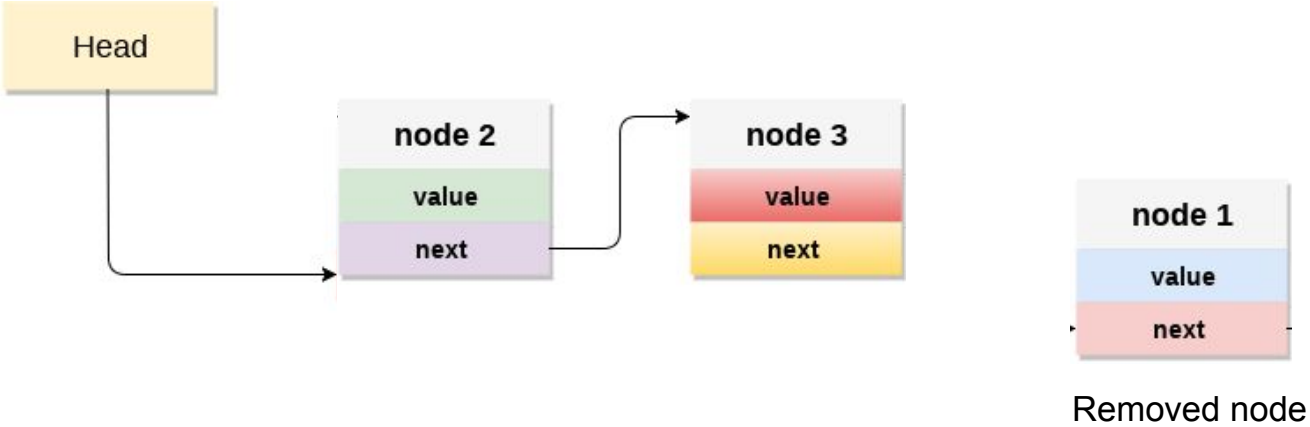
- Initialise a node
- Update the value of that node by data i.e. node.value = data
- Now link this node to the top of the linked list i.e. node.next = head
- And update top pointer to the current node i.e. head = node



A2SV
Africa To Silicon Valley

POP operation

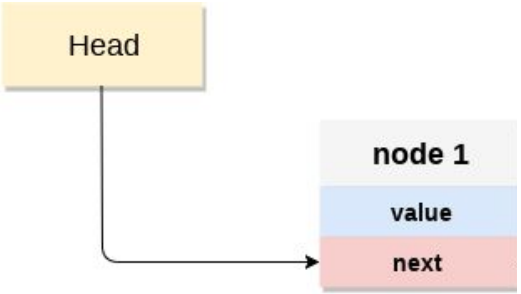
- First Check whether there is any node present in the linked list or not, if not then return
- Otherwise make pointer let say temp point to the top node and move forward the top node by 1 step
- Now free this temp node



A2SV
Africa To Silicon Valley

Top operation

- Check if there is any node present or not, if not then return.
- Otherwise return the value of top node of the linked list which is the node at **Head**



A2SV
Africa To Silicon Valley

Implementation

```python
class Node:
    def __init__(self, data):
        self.data = data
        self.next = None

class Stack:
    def __init__(self):
        self.head = None

    def isempty(self):
        if self.head == None:
            return True
        else:
            return False

    def peek(self):
        if self.isempty():
            return None
        else:
            return self.head.data
```

```python
def push(self, data):
##G5C is great
    if self.head == None:
        self.head = Node(data)
    else:
        new_node = Node(data)
        new_node.next = self.head
        self.head = new_node

def pop(self):

    if self.isempty():
        return None
    else:
        popped_node = self.head
        self.head = self.head.next
        popped_node.next = None
        return popped_node.data
```

A2SV
Africa To Silicon Valley

Time and space complexity

- Push
  - Time complexity - ___?
- Pop
  - Time complexity - ___?
- Peek
  - Time complexity - ___?
- isEmpty()
  - Time complexity - ___?

A2SV
Africa To Silicon Valley

Time and space complexity

- Push
  - Time complexity - O(1)
- Pop
  - Time complexity - O(1)
- Peek
  - Time complexity - O(1)
- isEmpty()
  - Time complexity - O(1)

A2SV
Africa To Silicon Valley

Applications of Stack

A2SV
Africa To Silicon Valley

Practice

Problem

A2SV
Africa To Silicon Valley

Solution

```python
class Solution:
    def isValid(self, s: str) -> bool:
        stack = []
        my_dict = {"(" : ")", "{" : "}", "[" : "]"}
        for i in range(len(s)):
            if s[i] in my_dict.keys():
                stack.append(s[i])
            else:
                if not stack:
                    return False
                a = stack.pop()
                if s[i] != my_dict[a]:
                    return False
        return stack == []
```

A2SV
Africa To Silicon Valley

Practice

Problem

A2SV
Africa To Silicon Valley

Solution

```python
class Solution:
    def removeStars(self, s: str) -> str:
        N = len(s)
        stack = []
        for i in range(N):
            if s[i] == "*":
                if stack:
                    stack.pop()
            else:
                stack.append(s[i])
        return ''.join(stack)
```

A2SV
Africa To Silicon Valley

Reflection: Stack can help you simulate deletion of elements in the middle in O(1) time complexity

A2SV
Africa To Silicon Valley

Pair Programming

Problem

A2SV
Africa To Silicon Valley

Solution

```python
class Solution:
    def minOperations(self, logs: List[str]) -> int:
        stack = []
        for log in logs:
            if log == '../':
                if stack:
                    stack.pop()
            elif log == './':
                continue
            else:
                stack.append(log)
        return len(stack)
```

A2SV
Africa To Silicon Valley

Reflection: stack can help you defer decision until some tasks are finished.

The bottom of the stack waits on the top of stack until they are processed

A2SV
Africa To Silicon Valley

Common PitFalls

- Popping from empty list
  - This will throw index out of range error
- Null pointer exception if we are using linked list

Runtime Error

```
IndexError: list index out of range
    if i == open_close[stack[-1]]:
Line 6 in isValid (Solution.py)
    ret = Solution().isValid(param_1)
Line 32 in _driver (Solution.py)
    _driver()
Line 43 in <module> (Solution.py)
```

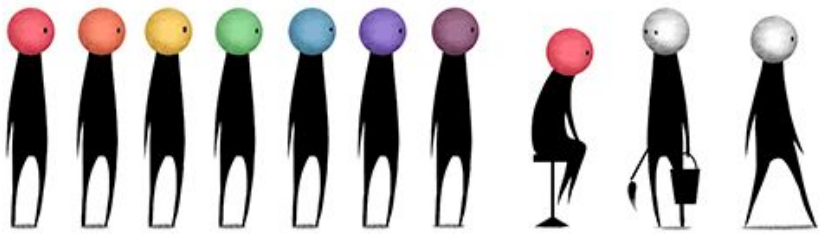
A2SV
Africa To Silicon Valley

Common PitFalls

- Stack overflow
  - May be not in python but In other programming language
  - Pushing to a full stack

A2SV
Africa To Silicon Valley

Queues

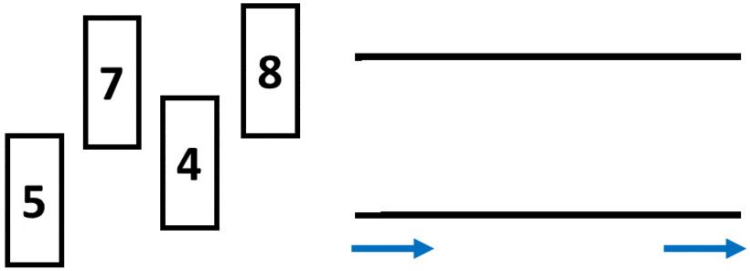


A2SV
Africa To Silicon Valley

Introduction

A collection whose elements are added at one end (the **rear**) and removed from the other end (the **front**)

- Uses **FIFO** data handling

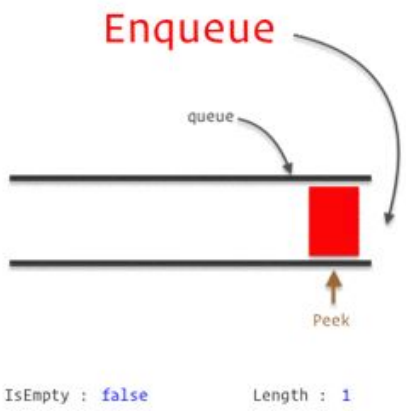


A2SV
Africa To Silicon Valley

Real Life Example

A2SV
Africa To Silicon Valley

Queue Operations

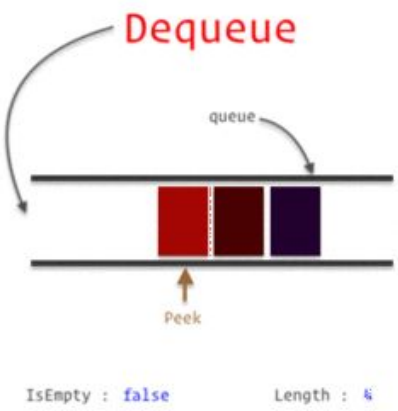


Enqueue (Append)

- Add an element to the tail of a queue
- **F**irst **I**n

A2SV
Africa To Silicon Valley

Queue Operations



Dequeue (Popleft)

- Remove an element from the head of the queue
- **F**irst **O**ut

A2SV
Africa To Silicon Valley

Practice

Problem

A2SV
Africa To Silicon Valley

Solution

```python
class RecentCounter:
    def __init__(self):
        self.queue = []

    def ping(self, t: int) -> int:
        self.queue.append(t)
        while (t - self.queue[0]) > 3000:
            self.queue.pop(0)
        return len(self.queue)
```

A2SV
Africa To Silicon Valley

Implementing Queue

Using an array to implement a queue is significantly harder than using an array to implement a stack. **Why**?

What would the time complexity be?

A2SV
Africa To Silicon Valley

Implementing Queue with List

```
def __init__(self):
   self.queue = []
   self.headIndex = 0

def append(self, value: int):
   self.queue.append(value)

def pop(self) -> int:
   if self.headIndex < len(self.queue):
        val = self.queue[self.headIndex]
        self.headIndex += 1
        return val
```

A2SV
Africa To Silicon Valley

A More Optimal Implementation

```
def __init__(self, size: int):
    self.queue = [None] * size
    self.headIndex = 0
    self.tailIndex = 0
    self.size = size
    self.count = 0

def append(self, value: int) -> bool:
    if self.count == self.size:
        return False
    self.queue[self.tailIndex] = value
    self.tail = (self.tailIndex + 1) % self.size
    self.count += 1
    return True
```

A2SV
Africa To Silicon Valley

Continued ...

```
def pop(self) -> int:
    if self.count == 0:
        None
    value = self.queue[self.headIndex]
    self.queue[self.headIndex] = None
    self.headIndex = (self.headIndex + 1) % self.size
    self.count -= 1
    return value
```

A2SV
Africa To Silicon Valley

Implementing Queue

- Either linked list or list can be used with careful considerations
- In practice, prefer to use built-in or library implementations like `deque()`
- Internally, `deque()` is implemented as a linked list of nodes

```
.pop()
.append()
.popleft()
.appendleft()
```

A2SV
Africa To Silicon Valley

Implementation (built-in)

```python
from collections import deque
# Initializing a queue
queue = deque()
# Adding elements to a queue
queue.append('a')
queue.append('b')
# Removing elements from a queue
print(queue.popleft())
print(queue.popleft())
# Uncommenting queue.popleft()
# will raise an IndexError
# as queue is now empty
```

We can also use it the other way around by using;

- `.appendleft()`
- `.pop()`

A2SV
Africa To Silicon Valley

Time and space complexity

- Append
  - Time complexity - ____?
- Popleft
  - Time complexity - ____?
- Peek
  - Time complexity - ____?
- isEmpty()
  - Time complexity - ____?

A2SV
Africa To Silicon Valley

Time and space complexity

- Append
  - Time complexity - O(1)
- Popleft
  - Time complexity - O(1)
- Peek
  - Time complexity - O(1)
- isEmpty()
  - Time complexity - O(1)

A2SV
Africa To Silicon Valley

Applications of Queue

A2SV
Africa To Silicon Valley

Practice

Problem

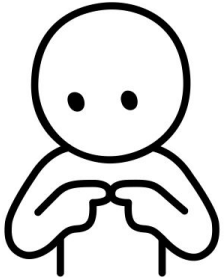
A2SV
Africa To Silicon Valley

Solution

```python
from collections import deque
class DataStream:
    def __init__(self, value: int, k: int):
        self.value = value
        self.k = k
        self.deque = deque()
        self.count = 0

    def consec(self, num: int) -> bool:
        if len(self.deque) == self.k:
            if self.deque[0] == self.value:
                self.count -= 1
            self.deque.popleft()

        self.deque.append(num)
        if num == self.value:
            self.count += 1

        return self.count == self.k
```

A2SV
Africa To Silicon Valley

Reflection: Queue helps solve problems that need access to the “first something”



Common Pitfalls

A2SV
Africa To Silicon Valley

Not handling edge cases

- Popping from an empty queue
  - ```
    if queue:
        queue.popleft()
    ```
- Appending to a full queue
  - ```
    if len(queue) < capacity:
        queue.append(val)
    ```

A2SV
Africa To Silicon Valley

Practice Questions

Stacks

- Valid Parentheses
- Simplify Path
- Evaluate Reverse Polish Notation
- Score of parenthesis
- Backspace String Compare

Queues

- Number of recent calls
- Find consecutive integers
- Design Circular Deque
- Implement Queue using Stack
- Shortest subarray with sum at least K

Monotonic

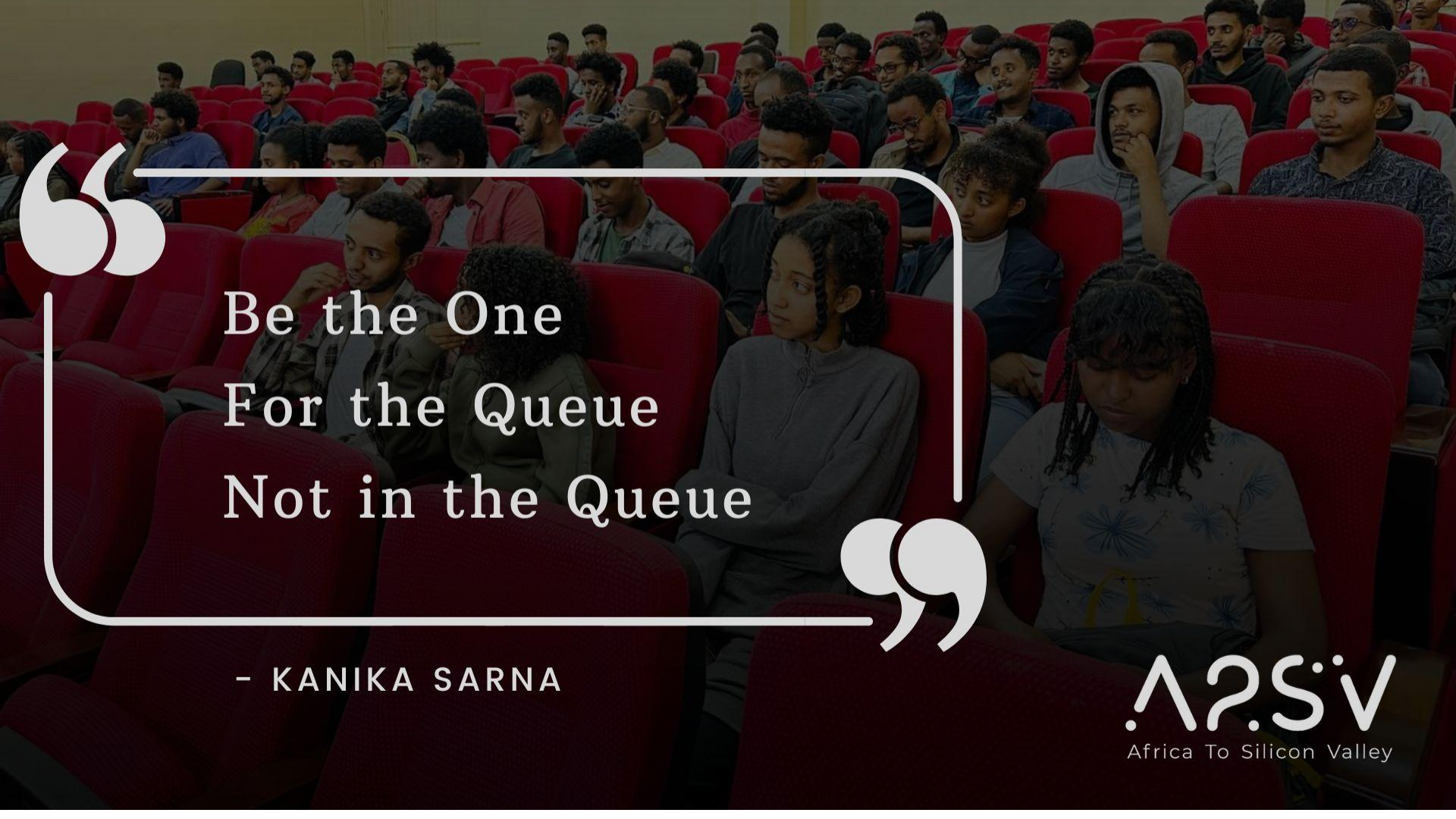
- Car Fleet
- Remove duplicates
- Sum of subarray minimum
- Remove k digits
- 132 Pattern

A2SV
Africa To Silicon Valley

Resources

A comprehensive guide and template for monotonic stack based problems

A2SV
Africa To Silicon Valley


Be the One
For the Queue
Not in the Queue
- KANIKA SARNA
A2SV
Africa To Silicon Valley